

Part 1 — Current Status

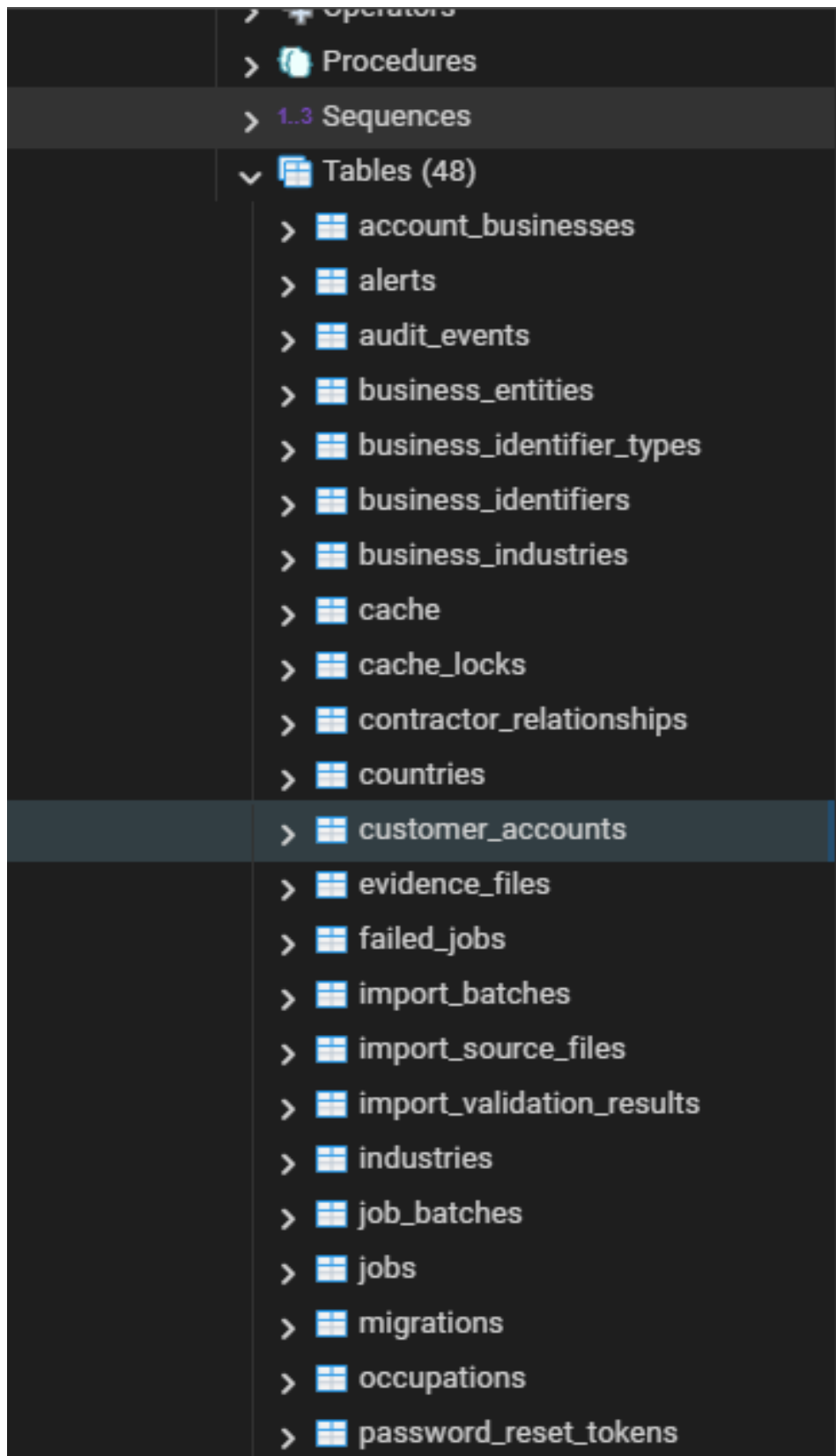
```
D:\laragon\www\opsfortress-demo-refactor
^ php artisan migrate:status

Migration name ..... Batch / Status
0001_01_01_000000_create_users_table ..... [1] Ran
0001_01_01_000001_create_cache_table ..... [1] Ran
0001_01_01_000002_create_jobs_table ..... [1] Ran
2025_08_14_170933_add_two_factor_columns_to_users_table ..... [1] Ran
2026_04_28_212300_create_core_platform_tables ..... [1] Ran
2026_04_28_212400_add_platform_columns_to_users_table ..... [1] Ran
2026_04_28_212500_create_workflow_tables ..... [1] Ran
2026_05_12_100000_create_audit_events_table ..... [1] Ran
2026_05_12_100100_fix_nullable_unique_assignment_indexes ..... [1] Ran
2026_05_12_100200_fix_file_uploads_disk_default ..... [1] Ran
2026_05_12_110000_add_person_type_to_users_table ..... [1] Ran
2026_05_14_100000_fix_task_packs_code_uniqueness ..... [1] Ran
2026_05_14_100100_widen_blockchain_id_and_abn_unique ..... [1] Ran
2026_05_18_000000_enable_postgres_extensions_and_prepare_v0_3_reset ..... [1] Ran
2026_05_18_000001_create_platform_lookup_tables ..... [1] Ran
2026_05_18_000002_create_customer_account_and_business_tables ..... [1] Ran
2026_05_18_000003_create_users_and_access_tables ..... [1] Ran
2026_05_18_000004_create_contractor_relationships ..... [1] Ran
2026_05_18_000005_create_whs_master_content_tables ..... [1] Ran
2026_05_18_000006_create_swms_content_tables ..... [1] Ran
2026_05_18_000007_create_import_tracking_tables ..... [1] Ran
2026_05_18_000008_create_runtime_tables ..... [1] Ran
2026_05_18_000009_create_evidence_audit_alert_tables ..... [1] Ran
2026_05_18_000010_create_p1_posttask_tables ..... [1] Ran
2026_05_18_000011_create_p1_training_tables ..... [1] Ran

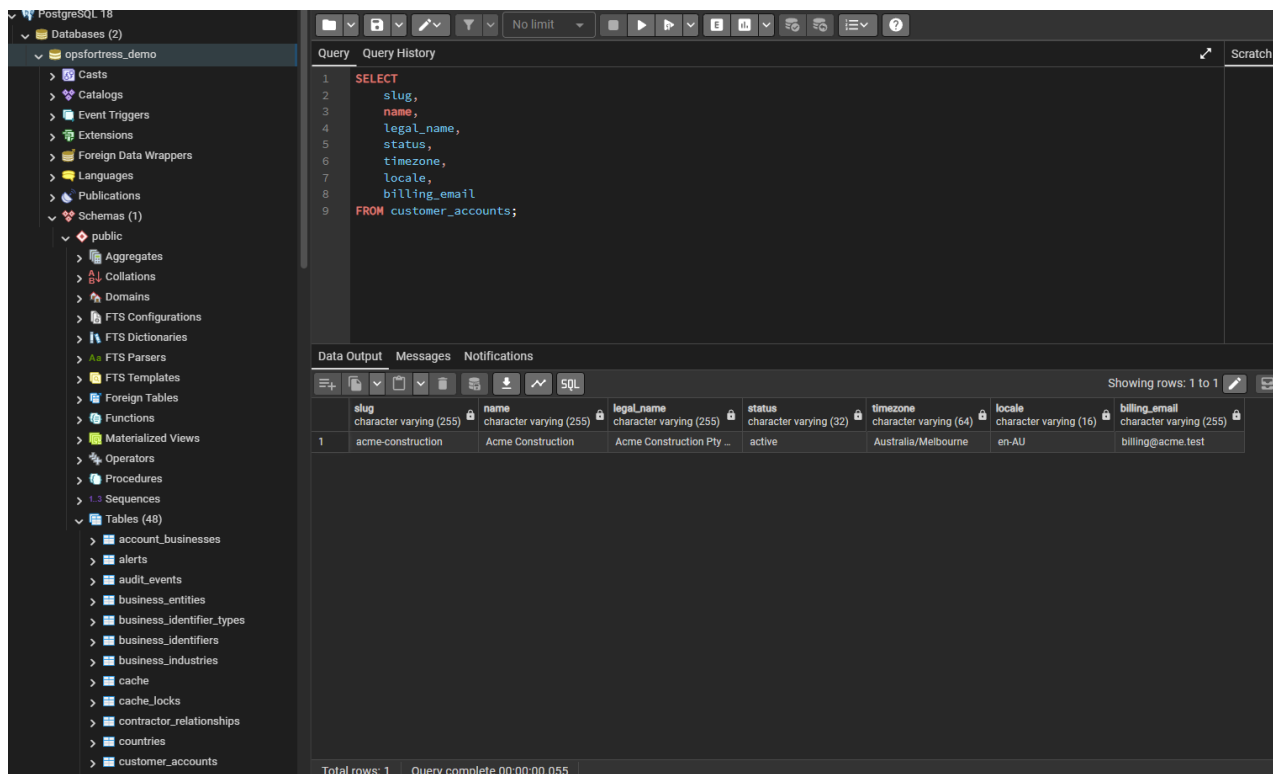
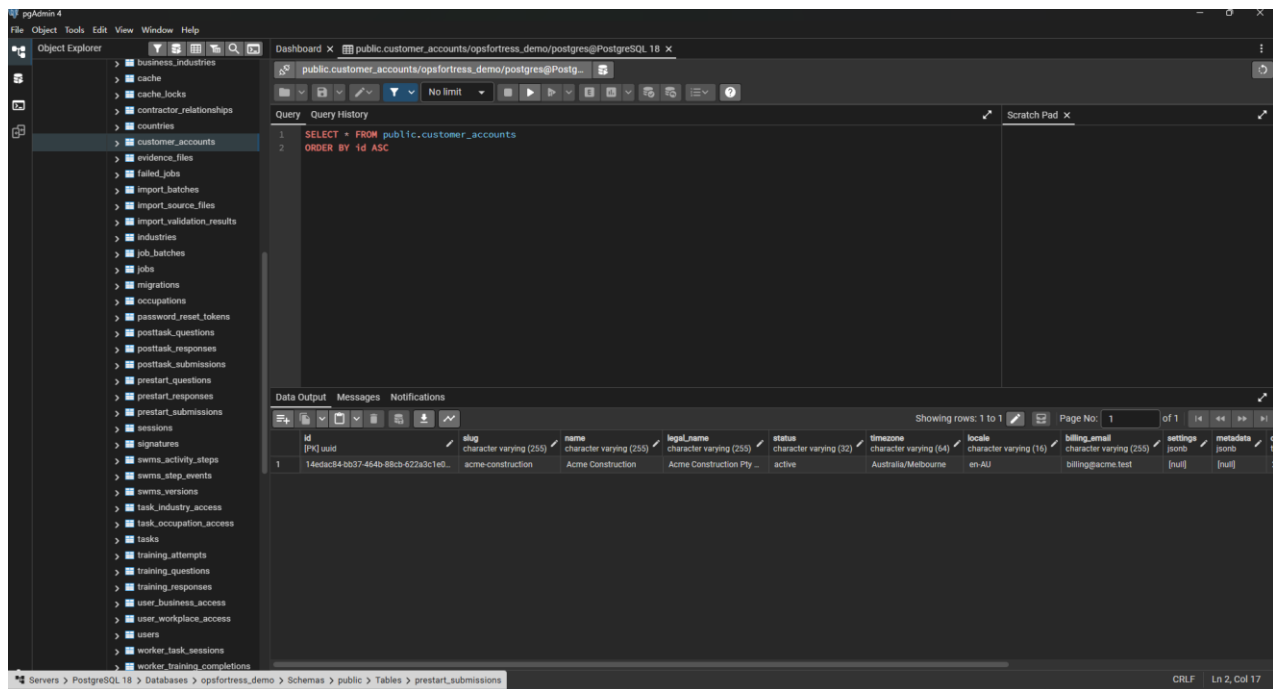
D:\laragon\www\opsfortress-demo-refactor
^ |
```

This screenshot confirms that all Laravel migrations have been executed successfully.

Status = Ran



The migration created 48 tables in PostgreSQL.



This screenshot clearly confirms the following:

The customer_accounts table contains seed data.

Demo account = Acme Construction

Status = active

Timezone = Australia/Melbourne

Locale = en-AU

Billing email = billing@acme.test

```
Query
Query History
1 SELECT 'customer_accounts' AS table_name, COUNT(*) FROM customer_accounts
2 UNION ALL
3 SELECT 'business_entities', COUNT(*) FROM business_entities
4 UNION ALL
5 SELECT 'workplaces', COUNT(*) FROM workplaces
6 UNION ALL
7 SELECT 'users', COUNT(*) FROM users
8 UNION ALL
9 SELECT 'tasks', COUNT(*) FROM tasks
10 UNION ALL
11 SELECT 'swms_versions', COUNT(*) FROM swms_versions
12 UNION ALL
13 SELECT 'swms_activity_steps', COUNT(*) FROM swms_activity_steps
14 UNION ALL
15 SELECT 'prestart_questions', COUNT(*) FROM prestart_questions;
```

Data Output Messages Notifications

Showing rows: 1 to 8

	table_name text	count bigint
1	customer_accounts	1
2	business_entities	1
3	workplaces	1
4	users	1
5	tasks	1
6	swms_versions	1
7	swms_activity_ste...	1
8	prestart_questions	1

This confirms that the key tables in the core demo chain contain seed data:

customer_accounts	1	Demo customer account exists
business_entities	1	Demo legal business exists
workplaces	1	Demo workplace exists
users	1	Demo user exists
tasks	1	Demo task exists
swms_versions	1	Demo SWMS version exists
swms_activity_steps	1	Demo worker step exists
prestart_questions	1	Demo pre-start question exists

The local PostgreSQL and Laravel environment has been successfully prepared for database review and validation.

A clean PostgreSQL database named `opsfortress_demo` was created locally. The latest Laravel refactor migration was executed successfully through `php artisan migrate --seed`. The migration created 48 tables in the public schema, and the database seeder inserted demo data into the key core tables.

The current local database is now suitable for DA/BA review of the v0.3 database structure, including table coverage, key relationships, seed data sanity, and importer readiness.

Evidence collected:

- Laravel migration status shows migrations have run successfully.

- pgAdmin shows 48 tables created in the opsfortress_demo database.
- customer_accounts contains demo seed data for Acme Construction.
- Core table row count query confirms demo rows exist across the main customer, business, workplace, user, task, SWMS, activity step, and pre-start question tables.

Initial observation:

The current seed data is minimal and mainly confirms that the database structure and seeding process work. It should not be treated as production-level SWMS content volume. For production import validation, we still need to test realistic workbooks with full activity steps, pre-start questions, occupation/industry access, and SWMS content.

Part 2 — Database Scope Overview

QueryQuery History

1234567

SELECT

table_name

FROM information_schema.tables

WHERE table_schema = 'public'

AND table_type = 'BASE TABLE'

ORDER BY table_name;

Data OutputMessagesNotifications

≡+

SQL

	table_name	
	name	
1	account_businesses	
2	alerts	
3	audit_events	
4	business_entities	
5	business_identifier_types	
6	business_identifiers	
7	business_industries	
8	cache	
9	cache_locks	
10	contractor_relationships	
11	countries	
12	customer_accounts	
13	evidence_files	
14	failed_jobs	
15	import_batches	
16	import_source_files	
17	import_validation_results	
18	industries	
19	job_batches	
20	jobs	
21	migrations	
22	occupations	
23	password_reset_tokens	

Total rows: 48

Query complete 00:00:00.042

Based on the current migration structure, the 48 tables can be grouped into the following functional areas.

A. Laravel system tables

These tables are required by the Laravel framework itself. They are not OpsFortress business data tables.

cache
cache_locks
failed_jobs
job_batches
jobs
migrations
password_reset_tokens
sessions

These tables are Laravel framework/system tables used for cache, queues, sessions, password reset, jobs and migration tracking. They are not part of the WHS business data model.

B. Core Platform / Onboarding tables

These tables belong to the OpsFortress core platform layer. They support accounts, businesses, workplaces, and user access.

customer_accounts
business_entities
account_businesses
business_identifiers
business_identifier_types
countries
workplaces
workplace_environments
users
user_business_access
user_workplace_access
contractor_relationships

Business meaning:

customer_accounts = paying / customer account layer

business_entities = legal business entities

account_businesses = account ↔ business relationship

business_identifiers = ABN, NZBN, EIN, and other business identifier values

workplaces = workplaces/sites under a business entity

users = people / login users

user_business_access = user access to a business entity

user_workplace_access = user access to a workplace/site

contractor_relationships = relationship between a host business and a contractor business

These tables form the OpsFortress core platform and onboarding structure. They support the relationship between customer accounts, legal business entities, workplaces, users, access permissions, business identifiers and contractor relationships.

C. WHS Content / Imported Master Data tables

These are the static content tables that the future importer will mainly write into.

industries
occupations
business_industries
tasks
task_occupation_access
task_industry_access
swms_versions
swms_activity_steps
prestart_questions
workplace_task_settings

industries / occupations = industry and occupation master data

tasks = SWMS/SOP task

swms_versions = versioned SWMS content

swms_activity_steps = worker-facing activity steps

prestart_questions = pre-start checklist questions

task_occupation_access = which occupations can view or perform a task

task_industry_access = which industries can view or use a task

workplace_task_settings = workplace-level task frequency/settings

These tables represent WHS task content and imported master data. They are the main target area for the future workbook importer, especially for tasks, SWMS versions, worker activity steps, pre-start questions, industries, occupations and access rules.

D. Runtime / Worker Action tables

These tables store data created when workers actually use the app. They should not be imported directly from Excel.

worker_task_sessions
swms_step_events
prestart_submissions
prestart_responses

Business meaning:

worker_task_sessions = one session where a worker starts or performs a task

swms_step_events = events showing that a worker viewed or confirmed SWMS steps

prestart_submissions = one submitted pre-start checklist

prestart_responses = answers to individual pre-start questions

These tables store worker runtime activity. They should be generated by the application when workers use the system, rather than imported directly from source workbooks.

E. Evidence / Audit / Alert tables

These tables support evidence, signatures, audit records, and alerts.

signatures
evidence_files
audit_events
alerts

signatures = digital signatures

evidence_files = photo/file evidence

audit_events = traceable audit log / hash-chain foundation

alerts = critical fail / escalation alerts

These tables support compliance evidence, traceability and alert handling. They are critical for proving worker acknowledgement, evidence capture and audit readiness.

F. Importer Tracking tables

These tables store records created by the importer itself during an import run.

import_batches
import_source_files
import_validation_results

Business meaning: import_batches = each import run

import_source_files = each source workbook or file processed by the importer

import_validation_results = validation pass/fail results

These tables show that the database is prepared for importer tracking. They allow each import batch, source file and validation result to be recorded. The actual workbook importer service still needs to be implemented or confirmed.

G. P1 Extension tables

These tables already exist, but can be treated as P1 / next-phase scope.

posttask_questions
posttask_submissions
posttask_responses
training_questions
training_attempts
training_responses
worker_training_completions

posttask = task completion / post-task check

training = periodic training assessment / completion records

These tables support P1 post-task and training flows. They are present in the schema, but should be treated as secondary unless the first importer/demo scope requires the full pre-start, post-task and training workflow.

Source Workbook / Python-generated Excel



Importer Tracking



WHS Content Tables



Worker App Runtime Tables



Evidence + Audit Trail

Part 3 — P0 / P1 Table Coverage Review

P0 should cover five areas:

1. Core Platform / Onboarding
2. Business / Workplace / User Access
3. WHS Content / Imported Master Data
4. Worker Runtime
5. Evidence / Audit / Import Tracking

I ran a SQL query to check whether the expected P0 and P1 tables exist.

Query	Query History
20	('P0 WHS Content / Imported Master Data', 'prestart_questions'),
21	('P0 WHS Content / Imported Master Data', 'industries'),
22	('P0 WHS Content / Imported Master Data', 'occupations'),
23	('P0 WHS Content / Imported Master Data', 'business_industries'),
24	('P0 WHS Content / Imported Master Data', 'task_occupation_access'),
25	('P0 WHS Content / Imported Master Data', 'task_industry_access'),
26	('P0 WHS Content / Imported Master Data', 'workplace_task_settings'),
27	
28	('P0 Runtime / Worker Actions', 'worker_task_sessions'),
29	('P0 Runtime / Worker Actions', 'swms_step_events'),
30	('P0 Runtime / Worker Actions', 'prestart_submissions'),
31	('P0 Runtime / Worker Actions', 'prestart_responses'),
32	
33	('P0 Evidence / Audit / Import Tracking', 'signatures'),
34	('P0 Evidence / Audit / Import Tracking', 'evidence_files'),
35	('P0 Evidence / Audit / Import Tracking', 'audit_events'),
36	('P0 Evidence / Audit / Import Tracking', 'alerts'),
37	('P0 Evidence / Audit / Import Tracking', 'import_batches'),
38	('P0 Evidence / Audit / Import Tracking', 'import_source_files'),
39	('P0 Evidence / Audit / Import Tracking', 'import_validation_results'),
40	
41	('P1 Post-task / Training', 'posttask_questions'),
42	('P1 Post-task / Training', 'posttask_submissions'),
43	('P1 Post-task / Training', 'posttask_responses'),
44	('P1 Post-task / Training', 'training_questions'),
45	('P1 Post-task / Training', 'training_attempts'),
46	('P1 Post-task / Training', 'training_responses'),
47	('P1 Post-task / Training', 'worker_training_completions')
48	)
49	SELECT
50	e.layer,
51	e.table_name,
52	CASE
53	WHEN t.table_name IS NOT NULL THEN 'Exists'
54	ELSE 'Missing'
55	END AS status
56	FROM expected_tables e
57	LEFT JOIN information_schema.tables t
58	ON t.table_schema = 'public'
59	AND t.table_type = 'BASE TABLE'
60	AND t.table_name = e.table_name
61	ORDER BY e.layer, e.table_name;
Total rows: 40	Query complete 00:00:00.033

This SQL produces a review table with:

layer	table_name	status
P0 Core / Onboarding	account_businesses	Exists
P0 Core / Onboarding	business_entities	Exists
P0 Core / Onboarding	business_identifier_types	Exists
P0 Core / Onboarding	business_identifiers	Exists
P0 Core / Onboarding	countries	Exists
P0 Core / Onboarding	customer_accounts	Exists
P0 Core / Onboarding	workplace_environments	Exists
P0 Core / Onboarding	workplaces	Exists

P0 Evidence / Audit / Import Tracking	alerts	Exists
P0 Evidence / Audit / Import Tracking	audit_events	Exists
P0 Evidence / Audit / Import Tracking	evidence_files	Exists
P0 Evidence / Audit / Import Tracking	import_batches	Exists
P0 Evidence / Audit / Import Tracking	import_source_files	Exists
P0 Evidence / Audit / Import Tracking	import_validation_results	Exists
P0 Evidence / Audit / Import Tracking	signatures	Exists
P0 Runtime / Worker Actions	prestart_responses	Exists
P0 Runtime / Worker Actions	prestart_submissions	Exists
P0 Runtime / Worker Actions	swms_step_events	Exists
P0 Runtime / Worker Actions	worker_task_sessions	Exists
P0 User / Access	contractor_relationships	Exists
P0 User / Access	user_business_access	Exists
P0 User / Access	user_workplace_access	Exists
P0 User / Access	users	Exists
P0 WHS Content / Imported Master Data	business_industries	Exists
P0 WHS Content / Imported Master Data	industries	Exists
P0 WHS Content / Imported Master Data	occupations	Exists
P0 WHS Content / Imported Master Data	prestart_questions	Exists
P0 WHS Content / Imported Master Data	swms_activity_steps	Exists
P0 WHS Content / Imported Master Data	swms_versions	Exists
P0 WHS Content / Imported Master Data	task_industry_access	Exists
P0 WHS Content / Imported Master Data	task_occupation_access	Exists
P0 WHS Content / Imported Master Data	tasks	Exists
P0 WHS Content / Imported Master Data	workplace_task_settings	Exists
P1 Post-task / Training	posttask_questions	Exists
P1 Post-task / Training	posttask_responses	Exists
P1 Post-task / Training	posttask_submissions	Exists
P1 Post-task / Training	training_attempts	Exists
P1 Post-task / Training	training_questions	Exists
P1 Post-task / Training	training_responses	Exists
P1 Post-task / Training	worker_training_completions	Exists

Summary Table:

Area	Coverage Status	Review Comment
P0 Core / Onboarding	Covered	Account, business, workplace and identifier structure exists
P0 User / Access	Covered	User, business access, workplace access and contractor relationship tables exist
P0 WHS Content / Imported Master Data	Covered	Main importer target tables exist
P0 Runtime / Worker Actions	Covered	Worker-generated session, step event and pre-start response tables exist

Area	Coverage Status	Review Comment
P0 Evidence / Audit / Import Tracking	Covered	Evidence, audit, alerts and importer tracking tables exist
P1 Post-task / Training	Present	Tables exist, but first-stage usage needs confirmation

The P0/P1 coverage check confirms that all expected tables are present in the current v0.3 migrated database.

All P0 Core / Onboarding, User / Access, WHS Content / Imported Master Data, Runtime / Worker Action, Evidence / Audit / Import Tracking tables exist. This indicates that the current migration covers the main database foundation required for the first OpsFortress / WHS App workflow.

The P1 Post-task / Training tables also exist. These tables provide support for the broader workflow, but their first-stage usage should still be confirmed with Kevin and Yiming before they are included in the first importer or demo scope.

Coverage is good, but coverage does not yet prove relationship correctness. The next step is to validate the foreign key relationships and confirm that the schema logic matches the expected business workflow.

Part 4 — Core Relationship Validation

The relationship review validates three main chains:

1. Core Platform relationship

Customer Account → Business Entity → Workplace → User Access

2. WHS Content relationship

Task → SWMS Version → SWMS Activity Steps / Prestart Questions

3. Runtime / Evidence relationship

Worker Task Session → Responses / Signatures / Evidence / Audit

4.1 Foreign key overview

source_table	source_column	target_table	target_column	constraint_name
account_businesses	account_id	customer_accounts	id	account_businesses_account_id_foreign
account_businesses	business_entity_id	business_entities	id	account_businesses_business_entity_id_foreign
alerts	account_id	customer_accounts	id	alerts_account_id_foreign
alerts	assigned_to_user_id	users	id	alerts_assigned_to_user_id_foreign
alerts	business_entity_id	business_entities	id	alerts_business_entity_id_foreign
alerts	prestart_submission_id	prestart_submissions	id	alerts_prestart_submission_id_foreign
alerts	supersedes_id	alerts	id	alerts_supersedes_id_foreign
alerts	triggered_by_user_id	users	id	alerts_triggered_by_user_id_foreign
alerts	worker_task_session_id	worker_task_sessions	id	alerts_worker_task_session_id_foreign
alerts	workplace_id	workplaces	id	alerts_workplace_id_foreign
audit_events	account_id	customer_accounts	id	audit_events_account_id_foreign
audit_events	business_entity_id	business_entities	id	audit_events_business_entity_id_foreign
audit_events	user_id	users	id	audit_events_user_id_foreign
audit_events	workplace_id	workplaces	id	audit_events_workplace_id_foreign
business_entities	country_id	countries	id	business_entities_country_id_foreign
business_identifier_types	country_id	countries	id	business_identifier_types_country_id_foreign
business_identifiers	business_entity_id	business_entities	id	business_identifiers_business_entity_id_foreign
business_identifiers	identifier_type_id	business_identifier_types	id	business_identifiers_identifier_type_id_foreign
business_identifiers	verified_by_user_id	users	id	business_identifiers_verified_by_user_id_foreign
business_industries	account_id	customer_accounts	id	business_industries_account_id_foreign
business_industries	business_entity_id	business_entities	id	business_industries_business_entity_id_foreign
business_industries	industry_id	industries	id	business_industries_industry_id_foreign
contractor_relationships	account_id	customer_accounts	id	contractor_relationships_account_id_foreign

contractor_relationships	approved_by_user_id	users	id	contractor_relationships_approved_by_user_id_foreign
contractor_relationships	contractor_business_entity_id	business_entities	id	contractor_relationships_contractor_business_entity_id_foreign
contractor_relationships	host_business_entity_id	business_entities	id	contractor_relationships_host_business_entity_id_foreign
evidence_files	account_id	customer_accounts	id	evidence_files_account_id_foreign
evidence_files	business_entity_id	business_entities	id	evidence_files_business_entity_id_foreign
evidence_files	prestart_submission_id	prestart_submissions	id	evidence_files_prestart_submission_id_foreign
evidence_files	signature_id	signatures	id	evidence_files_signature_id_foreign
evidence_files	supersedes_id	evidence_files	id	evidence_files_supersedes_id_foreign
evidence_files	uploaded_by_user_id	users	id	evidence_files_uploaded_by_user_id_foreign
evidence_files	worker_task_session_id	worker_task_sessions	id	evidence_files_worker_task_session_id_foreign
evidence_files	workplace_id	workplaces	id	evidence_files_workplace_id_foreign
import_batches	account_id	customer_accounts	id	import_batches_account_id_foreign
import_batches	started_by_user_id	users	id	import_batches_started_by_user_id_foreign
import_source_files	import_batch_id	import_batches	id	import_source_files_import_batch_id_foreign
import_validation_results	import_batch_id	import_batches	id	import_validation_results_import_batch_id_foreign
import_validation_results	import_source_file_id	import_source_files	id	import_validation_results_import_source_file_id_foreign
import_validation_results	resolved_by_user_id	users	id	import_validation_results_resolved_by_user_id_foreign
posttask_questions	task_id	tasks	id	posttask_questions_task_id_foreign
posttask_responses	posttask_question_id	posttask_questions	id	posttask_responses_posttask_question_id_foreign
posttask_responses	posttask_submission_id	posttask_submissions	id	posttask_responses_posttask_submission_id_foreign
posttask_responses	supersedes_id	posttask_responses	id	posttask_responses_supersedes_id_foreign
posttask_responses	worker_task_session_id	worker_task_sessions	id	posttask_responses_worker_task_session_id_foreign
posttask_responses	worker_user_id	users	id	posttask_responses_worker_user_id_foreign
posttask_submissions	account_id	customer_accounts	id	posttask_submissions_account_id_foreign
posttask_submissions	business_entity_id	business_entities	id	posttask_submissions_business_entity_id_foreign
posttask_submissions	supersedes_id	posttask_submissions	id	posttask_submissions_supersedes_id_foreign
posttask_submissions	task_id	tasks	id	posttask_submissions_task_id_foreign
posttask_submissions	worker_task_session_id	worker_task_sessions	id	posttask_submissions_worker_task_session_id_foreign
posttask_submissions	worker_user_id	users	id	posttask_submissions_worker_user_id_foreign
posttask_submissions	workplace_id	workplaces	id	posttask_submissions_workplace_id_foreign
prestart_questions	task_id	tasks	id	prestart_questions_task_id_foreign
prestart_responses	prestart_question_id	prestart_questions	id	prestart_responses_prestart_question_id_foreign
prestart_responses	prestart_submission_id	prestart_submissions	id	prestart_responses_prestart_submission_id_foreign
prestart_responses	supersedes_id	prestart_responses	id	prestart_responses_supersedes_id_foreign

prestart_responses	worker_task_session_id	worker_task_sessions	id	prestart_responses_worker_task_session_id_foreign
prestart_responses	worker_user_id	users	id	prestart_responses_worker_user_id_foreign
prestart_submissions	account_id	customer_accounts	id	prestart_submissions_account_id_foreign
prestart_submissions	business_entity_id	business_entities	id	prestart_submissions_business_entity_id_foreign
prestart_submissions	supersedes_id	prestart_submissions	id	prestart_submissions_supersedes_id_foreign
prestart_submissions	task_id	tasks	id	prestart_submissions_task_id_foreign
prestart_submissions	worker_task_session_id	worker_task_sessions	id	prestart_submissions_worker_task_session_id_foreign
prestart_submissions	worker_user_id	users	id	prestart_submissions_worker_user_id_foreign
prestart_submissions	workplace_id	workplaces	id	prestart_submissions_workplace_id_foreign
signatures	account_id	customer_accounts	id	signatures_account_id_foreign
signatures	business_entity_id	business_entities	id	signatures_business_entity_id_foreign
signatures	supersedes_id	signatures	id	signatures_supersedes_id_foreign
signatures	user_id	users	id	signatures_user_id_foreign
signatures	worker_task_session_id	worker_task_sessions	id	signatures_worker_task_session_id_foreign
signatures	workplace_id	workplaces	id	signatures_workplace_id_foreign
swms_activity_steps	swms_version_id	swms_versions	id	swms_activity_steps_swms_version_id_foreign
swms_step_events	supersedes_id	swms_step_events	id	swms_step_events_supersedes_id_foreign
swms_step_events	swms_activity_step_id	swms_activity_steps	id	swms_step_events_swms_activity_step_id_foreign
swms_step_events	worker_task_session_id	worker_task_sessions	id	swms_step_events_worker_task_session_id_foreign
swms_step_events	worker_user_id	users	id	swms_step_events_worker_user_id_foreign
swms_versions	approved_by_user_id	users	id	swms_versions_approved_by_user_id_foreign
swms_versions	supersedes_id	swms_versions	id	swms_versions_supersedes_id_foreign
swms_versions	task_id	tasks	id	swms_versions_task_id_foreign
task_industry_access	industry_id	industries	id	task_industry_access_industry_id_foreign
task_industry_access	task_id	tasks	id	task_industry_access_task_id_foreign
task_occupation_access	occupation_id	occupations	id	task_occupation_access_occupation_id_foreign
task_occupation_access	task_id	tasks	id	task_occupation_access_task_id_foreign
training_attempts	account_id	customer_accounts	id	training_attempts_account_id_foreign
training_attempts	business_entity_id	business_entities	id	training_attempts_business_entity_id_foreign
training_attempts	supersedes_id	training_attempts	id	training_attempts_supersedes_id_foreign
training_attempts	swms_version_id	swms_versions	id	training_attempts_swms_version_id_foreign
training_attempts	task_id	tasks	id	training_attempts_task_id_foreign
training_attempts	worker_user_id	users	id	training_attempts_worker_user_id_foreign
training_attempts	workplace_id	workplaces	id	training_attempts_workplace_id_foreign
training_questions	swms_version_id	swms_versions	id	training_questions_swms_version_id_foreign
training_questions	task_id	tasks	id	training_questions_task_id_foreign
training_responses	supersedes_id	training_responses	id	training_responses_supersedes_id_foreign
training_responses	training_attempt_id	training_attempts	id	training_responses_training_attempt_id_foreign
training_responses	training_question_id	training_questions	id	training_responses_training_question_id_foreign
training_responses	worker_user_id	users	id	training_responses_worker_user_id_foreign
user_business_access	account_id	customer_accounts	id	user_business_access_account_id_foreign
user_business_access	business_entity_id	business_entities	id	user_business_access_business_entity_id_foreign

user_business_access	granted_by_user_id	users	id	user_business_access_granted_by_user_id_foreign
user_business_access	user_id	users	id	user_business_access_user_id_foreign
user_workplace_access	account_id	customer_accounts	id	user_workplace_access_account_id_foreign
user_workplace_access	business_entity_id	business_entities	id	user_workplace_access_business_entity_id_foreign
user_workplace_access	granted_by_user_id	users	id	user_workplace_access_granted_by_user_id_foreign
user_workplace_access	user_id	users	id	user_workplace_access_user_id_foreign
user_workplace_access	workplace_id	workplaces	id	user_workplace_access_workplace_id_foreign
users	account_id	customer_accounts	id	users_account_id_foreign
users	home_business_entity_id	business_entities	id	users_home_business_entity_id_foreign
worker_task_sessions	account_id	customer_accounts	id	worker_task_sessions_account_id_foreign
worker_task_sessions	business_entity_id	business_entities	id	worker_task_sessions_business_entity_id_foreign
worker_task_sessions	supersedes_id	worker_task_sessions	id	worker_task_sessions_supersedes_id_foreign
worker_task_sessions	swms_version_id	swms_versions	id	worker_task_sessions_swms_version_id_foreign
worker_task_sessions	task_id	tasks	id	worker_task_sessions_task_id_foreign
worker_task_sessions	worker_user_id	users	id	worker_task_sessions_worker_user_id_foreign
worker_task_sessions	workplace_id	workplaces	id	worker_task_sessions_workplace_id_foreign
worker_training_completions	account_id	customer_accounts	id	worker_training_completions_account_id_foreign
worker_training_completions	business_entity_id	business_entities	id	worker_training_completions_business_entity_id_foreign
worker_training_completions	latest_training_attempt_id	training_attempts	id	worker_training_completions_latest_training_attempt_id_foreign
worker_training_completions	task_id	tasks	id	worker_training_completions_task_id_foreign
worker_training_completions	worker_user_id	users	id	worker_training_completions_worker_user_id_foreign
worker_training_completions	workplace_id	workplaces	id	worker_training_completions_workplace_id_foreign
workplace_environments	workplace_id	workplaces	id	workplace_environments_workplace_id_foreign
workplace_task_settings	account_id	customer_accounts	id	workplace_task_settings_account_id_foreign
workplace_task_settings	active_swms_version_id	swms_versions	id	workplace_task_settings_active_swms_version_id_foreign
workplace_task_settings	business_entity_id	business_entities	id	workplace_task_settings_business_entity_id_foreign
workplace_task_settings	configured_by_user_id	users	id	workplace_task_settings_configured_by_user_id_foreign
workplace_task_settings	task_id	tasks	id	workplace_task_settings_task_id_foreign
workplace_task_settings	workplace_id	workplaces	id	workplace_task_settings_workplace_id_foreign
workplaces	account_id	customer_accounts	id	workplaces_account_id_foreign
workplaces	business_entity_id	business_entities	id	workplaces_business_entity_id_foreign
workplaces	country_id	countries	id	workplaces_country_id_foreign

(1) Core Platform / Onboarding relationship — Pass

From the foreign key table above, we can see the core account-to-business relationship.

This means one customer account can manage one or more legal business entities through `account_businesses`.

(2) Business Identifier relationship — Pass

This means each business entity can have identifiers such as ABN, NZBN, ACN, or EIN, and the identifier type is controlled through a standard lookup table.

(3) Business → Workplace relationship — Pass

The above table confirms the following relationship:

Customer Account

↓

Business Entity

↓

Workplace

This relationship is present and valid.

But I notice that:

`workplace_environments.workplace_id → workplaces.id`

This may mean that one workplace can have multiple environment records or classifications. This should be confirmed with Yiming: is `workplace_environments` intentionally designed as a child table of `workplaces`?

(4) User / Access relationship — Pass

The foreign key table shows that user access is not hard-coded directly inside the `users` table. Instead, access is controlled through access tables. A user can receive business-level access through `user_business_access` and workplace-level access through `user_workplace_access`.

(5) Contractor relationship — Pass

The foreign key export includes:

`contractor_relationships.host_business_entity_id → business_entities.id`

`contractor_relationships.contractor_business_entity_id → business_entities.id`

`contractor_relationships.account_id → customer_accounts.id`

contractor_relationships.approved_by_user_id → users.id

This is a strong design. It means a contractor is not simply placed under the host business. Instead, the contractor remains a separate business entity and is connected through a linked relationship. This matches the contractor architecture direction we discussed previously.

(6) WHS Content relationship — Pass

The core WHS content and access-mapping relationships are present:

swms_versions.task_id → tasks.id

swms_activity_steps.swms_version_id → swms_versions.id

prestart_questions.task_id → tasks.id

This means the main target area for the importer has the correct relationship foundation. This is one of the most important passes in Part 4.

(7) Runtime / Worker Action relationship — Pass

The worker runtime relationship chain exists. This supports the principle we have been using:

worker action → event record → dashboard metric

(8) Evidence / Signature / Alert relationship — Mostly Pass

The table shows that evidence, signatures, and alerts can be traced back to a worker session or a pre-start submission.

This is important for compliance traceability.

One point still needs confirmation: audit_events.

The audit_events table has the following foreign keys:

audit_events.account_id → customer_accounts.id

audit_events.business_entity_id → business_entities.id

audit_events.user_id → users.id

audit_events.workplace_id → workplaces.id

However, in this foreign key export, I did not see:

audit_events.worker_task_session_id → worker_task_sessions.id

For audit_events, is the table designed to use a generic auditable reference model, such as entity_type/entity_id, instead of a direct worker_task_session_id foreign key?

If not, for worker runtime audit events, we may need either worker_task_session_id or a clear entity_type/entity_id reference so each audit event can be traced back to the exact worker session.

Part 4.1 Conclusion:

In this section, I exported the database foreign keys. There are 131 relationships in total. This confirms that the current database is not just 48 isolated tables; it already has a well-connected relationship structure.

The core relationships broadly pass the review.

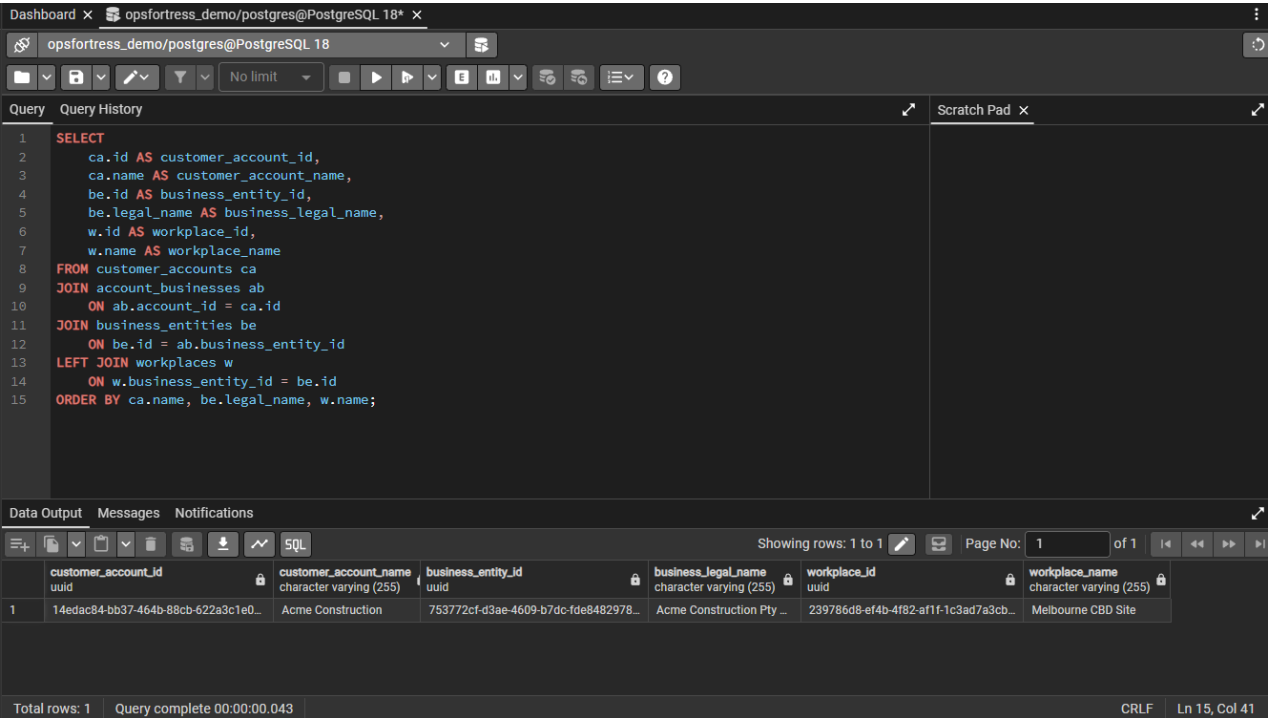
I identified two small points that should be confirmed:

First, workplace_environments appears to be a child table of workplaces. This may be intended to support multiple environment records for one workplace, but it should be confirmed with Yiming.

Second, audit_events does not show a direct worker_task_session_id foreign key. This may be because it uses a generic audit model, but it should also be confirmed.

Step 4.2 — Validate Seed Data Chain

Step 4.2A — Validate Customer → Business → Workplace



The screenshot shows a PostgreSQL query editor interface. The query is a SELECT statement that joins customer accounts, business entities, and workplaces. The results show a single row for 'Acme Construction' with its associated business entity and workplace.

```
1 SELECT
2   ca.id AS customer_account_id,
3   ca.name AS customer_account_name,
4   be.id AS business_entity_id,
5   be.legal_name AS business_legal_name,
6   w.id AS workplace_id,
7   w.name AS workplace_name
8 FROM customer_accounts ca
9 JOIN account_businesses ab
10  ON ab.account_id = ca.id
11 JOIN business_entities be
12  ON be.id = ab.business_entity_id
13 LEFT JOIN workplaces w
14  ON w.business_entity_id = be.id
15 ORDER BY ca.name, be.legal_name, w.name;
```

customer_account_id	customer_account_name	business_entity_id	business_legal_name	workplace_id	workplace_name
14edac84-bb37-464b-88cb-622a3c1e0...	Acme Construction	753772cf-d3ae-4609-b7dc-fde8482978...	Acme Construction Pty ...	239786d8-ef4b-4f82-af1f-1c3ad7a3cb...	Melbourne CBD Site

The customer → business → workplace relationship was validated through a join query. The seeded demo data successfully connects the Acme Construction customer account to the Acme Construction business entity and the Melbourne CBD Site workplace.

Validation result: Pass.

Step 4.2B — Validate User Access Chain

Query Query History

```
10 uwa.permission_role AS workplace_permission_role,
11 uwa.access_status AS workplace_access_status,
12 w.name AS workplace_name
13 FROM users u
14 LEFT JOIN user_business_access uba
15 ON uba.user_id = u.id
16 LEFT JOIN business_entities be
17 ON be.id = uba.business_entity_id
18 LEFT JOIN user_workplace_access uwa
19 ON uwa.user_id = u.id
20 LEFT JOIN workplaces w
21 ON w.id = uwa.workplace_id
22 ORDER BY u.name;
```

Scratch Pad

Data Output Messages Notifications

user_id	user_name	user_email	business_permission_role	business_access_status	business_legal_name	workplace_permission_role	workplace_access_status	workplace_name	
uid	character varying (255)	character varying (255)	character varying (32)	character varying (32)	character varying (255)	character varying (32)	character varying (32)	character varying (255)	
1	349d5a73-63ee-4da7-8105-fe474d678e...	Alex Admin	admin@acme.test	admin	active	Acme Construction Pty ...	admin	active	Melbourne CBD Site

The seeded demo user Alex Admin is linked to the Acme Construction business entity through user_business_access and to the Melbourne CBD Site workplace through user_workplace_access.

Validation result: Pass.

Technical note:

The access tables use `permission\_role` and `access\_status`, which separates the user's permission role from the current access state. This makes the permission model clearer.

Step 4.2C — Validate WHS Content

Task → SWMS Version → Activity Step / Prestart Question

Dashboard opsfortress_demo/postgres@PostgreSQL 18*

opsfortress_demo/postgres@PostgreSQL 18

No limit

Query Query History

```
1 SELECT
2   t.external_task_id,
3   t.task_name,
4   pq.question_number,
5   pq.prompt AS prestart_question_prompt,
6   pq.question_type,
7   pq.is_required,
8   pq.is_critical_failure
9 FROM tasks t
10 LEFT JOIN prestart_questions pq
11 ON pq.task_id = t.id
12 ORDER BY t.external_task_id, pq.question_number;
```

Data Output Messages Notifications

external_task_id	task_name	question_number	prestart_question_prompt	question_type	is_required	is_critical_failure	
character varying (255)	character varying (255)	smallint	text	character varying (32)	boolean	boolean	
1	TASK-DEMO-001	Daily Site Inspection	1	Are exclusion zones and access paths safe for work tod...	yes_no	true	true

This confirms that:

A task can successfully connect to a SWMS version, and the SWMS version can connect to an activity step.

The task → pre-start question relationship was validated through a join query. The demo task `TASK-DEMO-001` is linked to a pre-start question, and the question includes key validation fields such as question type, required flag and critical failure flag.

Validation result: Pass.

Note:

Current seed data contains only one pre-start question. This is enough to validate the relationship, but not enough to validate production quantity requirements. Full workbook validation should later confirm the expected number of pre-start questions, such as 15 questions if that remains the target format.

Step 4.2D — Worker Runtime Chain

The screenshot shows a PostgreSQL query editor interface. The query is a complex JOIN query that links various tables to validate the worker runtime chain. The query is as follows:

```
1 t.external_task_id,
2 t.task_name,
3 sv.version_label AS swms_version,
4 wts.status AS worker_session_status,
5 sse.event_type AS swms_step_event_type,
6 sas.step_number,
7 sas.title AS swms_step_title,
8 ps.status AS prestart_submission_status,
9 pq.question_number,
10 pr.answer_text,
11 pr.answer_boolean,
12 pr.is_critical_failure
13 FROM worker_task_sessions wts
14 LEFT JOIN users u
15 ON u.id = wts.worker_user_id
16 LEFT JOIN workplaces w
17 ON w.id = wts.workplace_id
18 LEFT JOIN tasks t
19 ON t.id = wts.task_id
20 LEFT JOIN swms_versions sv
21 ON sv.id = wts.swms_version_id
22 LEFT JOIN swms_step_events sse
23 ON sse.worker_task_session_id = wts.id
24 LEFT JOIN swms_activity_steps sas
25 ON sas.id = sse.swms_activity_step_id
26 LEFT JOIN prestart_submissions ps
27 ON ps.worker_task_session_id = wts.id
28 LEFT JOIN prestart_responses pr
29 ON pr.prestart_submission_id = ps.id
30 LEFT JOIN prestart_questions pq
31 ON pq.id = pr.prestart_question_id
32 ORDER BY wts.started_at DESC;
```

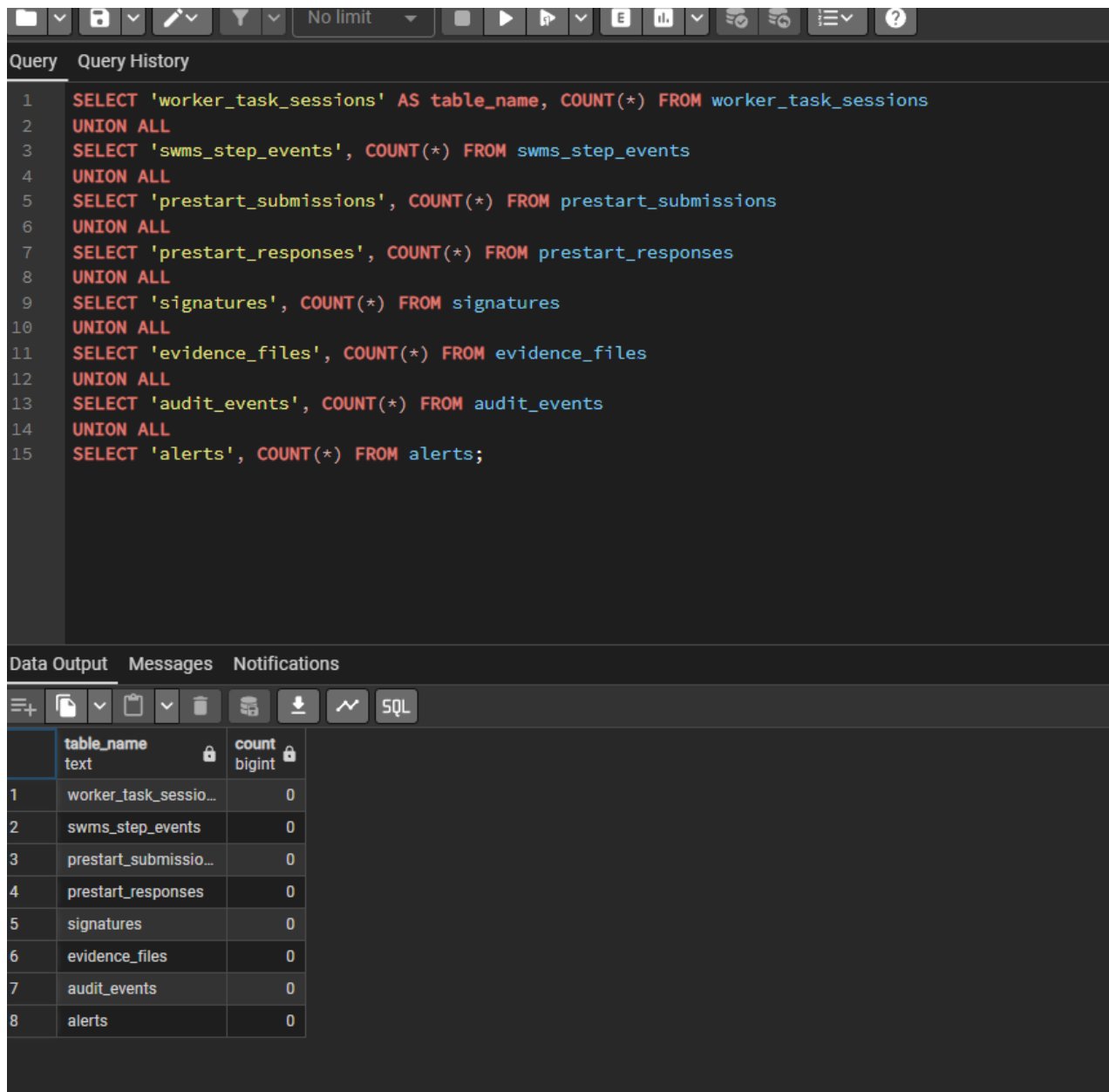
The data output shows the following columns and their data types:

worker_name	workplace_name	external_task_id	task_name	swms_version	worker_session_status	swms_step_event_type	step_number	swms_step_title	prestart_submission_status
character varying (255)	character varying (255)	character varying (255)	character varying (255)	character varying (64)	character varying (32)	character varying (32)	smallint	character varying (255)	character varying (32)

The runtime tables and foreign key relationships exist, but the current seed dataset does not contain a complete worker runtime session chain. Therefore, the worker runtime data flow should be re-tested once demo runtime records are added or once the front-end / worker app creates real session data.

Current seeding confirms the master/content setup, but it does not yet seed a full worker runtime journey. A small runtime seed should be added later so the team can review the complete app data flow.

Step 4.2E — Evidence / Signature / Audit Relationship Check



The screenshot shows a database query tool interface. The top toolbar includes icons for file operations, query execution, and settings. The main area displays a SQL query with line numbers 1 through 15. The query is a UNION ALL of SELECT statements, each counting records from a specific table. Below the query, the 'Data Output' tab is active, showing a table with two columns: 'table_name' (text) and 'count' (bigint). The table contains 8 rows, each corresponding to a table in the query. All counts are 0.

```
1 SELECT 'worker_task_sessions' AS table_name, COUNT(*) FROM worker_task_sessions
2 UNION ALL
3 SELECT 'swms_step_events', COUNT(*) FROM swms_step_events
4 UNION ALL
5 SELECT 'prestart_submissions', COUNT(*) FROM prestart_submissions
6 UNION ALL
7 SELECT 'prestart_responses', COUNT(*) FROM prestart_responses
8 UNION ALL
9 SELECT 'signatures', COUNT(*) FROM signatures
10 UNION ALL
11 SELECT 'evidence_files', COUNT(*) FROM evidence_files
12 UNION ALL
13 SELECT 'audit_events', COUNT(*) FROM audit_events
14 UNION ALL
15 SELECT 'alerts', COUNT(*) FROM alerts;
```

	table_name	count
1	worker_task_sessio...	0
2	swms_step_events	0
3	prestart_submissio...	0
4	prestart_responses	0
5	signatures	0
6	evidence_files	0
7	audit_events	0
8	alerts	0

The runtime and evidence tables currently contain no seed data. This means the current seeder confirms the master/content setup, but does not yet create a complete worker runtime journey.

The schema and foreign key structure for runtime records exists, but the actual worker action → event record → response/evidence/audit chain still needs to be tested later using either demo app activity or a dedicated runtime seeder.

Validation result:

- Runtime schema: Pass
- Runtime seed data chain: Not validated yet

Recommendation:

For future demo validation, add a small runtime seed chain:

demo worker → workplace → task → worker_task_session → SWMS step event → prestart submission → prestart response → signature/evidence/audit event.

Part 4 Conclusion:

The conclusion for Part 4 is that the core database relationships broadly pass the review. We validated that a customer account can connect to a business entity and workplace, a user can connect to a business and workplace through access tables, and a task can connect to a SWMS version, activity step, and pre-start question.

This confirms that the P0 master-data and content relationship structure is in place.

However, the current seed data does not include a full worker runtime journey, so a small demo runtime dataset should be added next.

Also, we should confirm with Yiming whether audit_events can be traced to a specific worker_task_session.

Part 5 — Imported Data vs App-Generated Data

This section is based on the Data Ownership / Row Count SQL result.

Dashboard x opsfortress_demo/postgres@PostgreSQL 18 x

opsfortress_demo/postgres@PostgreSQL 18

Query Query History Scratch Pad x

```
25 UNION ALL
26 SELECT 'App-generated runtime data', 'prestart_submissions', 'WHS App runtime', COUNT(*) FROM prestart_submissions
27 UNION ALL
28 SELECT 'App-generated runtime data', 'prestart_responses', 'WHS App runtime', COUNT(*) FROM prestart_responses
29 UNION ALL
30 SELECT 'App-generated runtime data', 'signatures', 'WHS App runtime', COUNT(*) FROM signatures
31 UNION ALL
```

Data Output Messages Notifications

Showing rows: 1 to 21 Page No: 1 of 1

	data_category	table_name	expected_created_by	current_rowcount
1	App-generated runtime data	alerts	WHS App runtime / system rules	0
2	App-generated runtime data	audit_events	WHS App runtime / system audit layer	0
3	App-generated runtime data	evidence_files	WHS App runtime	0
4	App-generated runtime data	prestart_responses	WHS App runtime	0
5	App-generated runtime data	prestart_submissions	WHS App runtime	0
6	App-generated runtime data	signatures	WHS App runtime	0
7	App-generated runtime data	swms_step_events	WHS App runtime	0
8	App-generated runtime data	worker_task_sessions	WHS App runtime	0
9	Imported content / master data	business_industries	Python-generated Excel / Laravel importer / admin se...	1
10	Imported content / master data	industries	Python-generated Excel / Laravel importer	1
11	Imported content / master data	occupations	Python-generated Excel / Laravel importer	1
12	Imported content / master data	prestart_questions	Python-generated Excel / Laravel importer	1
13	Imported content / master data	swms_activity_steps	Python-generated Excel / Laravel importer	1
14	Imported content / master data	swms_versions	Python-generated Excel / Laravel importer	1
15	Imported content / master data	task_industry_access	Python-generated Excel / Laravel importer	1
16	Imported content / master data	task_occupation_acce...	Python-generated Excel / Laravel importer	1
17	Imported content / master data	tasks	Python-generated Excel / Laravel importer	1
18	Imported content / master data	workplace_task_settin...	Laravel app / admin configuration	1
19	Importer-generated tracking d...	import_batches	Laravel importer	0
20	Importer-generated tracking d...	import_source_files	Laravel importer	0
21	Importer-generated tracking d...	import_validation_resu...	Laravel importer	0

Total rows: 21 Query complete 00:00:00.088 CRLF Ln 45, Col 36

This section confirms that the database separates the data into three clear groups.

First, the importer should bring in the fixed WHS/SWMS content. This includes tasks, SWMS versions, activity steps, pre-start questions, industries, occupations, and access rules. This is the type of data that should come from the Python-generated Excel workbook.

Second, the worker activity data should not come from Excel. This data should be created by the WHS App when workers actually use the system. This includes worker task sessions, SWMS step events, pre-start submissions, responses, signatures, evidence files, audit events, and alerts.

Third, the importer should create its own tracking records when it processes a workbook. This includes import batches, source files, and validation results. The current database result matches this logic:

- The imported content tables have minimal seed data.
- The worker runtime tables are currently empty.
- The importer tracking tables are currently empty.

This is the correct starting point. It means the database is ready to separate imported content, real worker activity, and importer validation records properly.

Part 6 — JSON Schema / Python / Importer Validation Requirements

The database review confirms that the future content workflow needs clear validation gates before data reaches PostgreSQL. The AI prompt should generate structured content, but JSON Schema, Python, and the Laravel importer should control structure, consistency, and database readiness.

6.1 Define the responsibility of each layer

Layer	Main Responsibility	Should Validate
Gold benchmark prompt	Generate WHS/SWMS content	Content quality, task wording, hazards, controls
Structured JSON	Hold AI output in machine-readable format	Task, activity, question, industry, occupation structure
JSON Schema	Check AI output before Python	Required fields, correct data types, valid structure
Python layer	Validate data and generate final Excel	Sheet names, columns, IDs, relationship rules
Importer-ready Excel	Final file submitted to importer	Fixed workbook framework
Laravel importer	Final check before database write	Required sheets, FK matching, duplicates, validation results
PostgreSQL	Store validated data	Final source of truth

6.2 What JSON Schema should validate

The JSON Schema should act as the first technical validation gate after the prompt. It should make sure the AI output is structured, complete, and machine-readable before it is passed into Python.

It should check:

task_id / external_task_id exists

task title exists

document_type is valid

SWMS/SOP content is structured

activity steps are structured

hazards and controls are structured

pre-start questions are structured

industry path exists

occupation / role path exists

required fields are present

data types are correct

6.3 What Python should validate

Python is the main structure-control layer. It is not only a data-cleaning step; it is responsible for producing the final standard Excel workbook.

Python should check:

fixed sheet names

fixed column names

fixed column order

required fields are not empty

ID format is correct

task IDs are unique

SWMS version links to a task

activity steps link to a SWMS version

pre-start questions link to a task

industry and occupation names can be mapped

yes/no and true/false values are standardised

allowed values are used

data types are correct

The main Python output should be a framework-compliant, importer-ready Excel workbook.

6.4 What the Laravel importer should validate

The importer should check:

required workbook sheets exist

required columns exist

column names match expected names

required fields are not blank

task IDs are not duplicated

foreign key targets can be resolved

industry / occupation mapping exists

invalid rows are rejected

validation results are recorded

failed validation blocks database import

successful import creates import_batches record

source workbook is recorded in import_source_files

errors are recorded in import_validation_results

The importer is the final gatekeeper. Even if Python has already validated the workbook, the Laravel importer should still run its own checks before writing to PostgreSQL.

6.5 Database-side validation requirements

Based on the database review, the following database-side rules should guide the JSON Schema, Python layer, and importer.

Requirement	Reason
Task must have a unique external ID	Needed to import into tasks
SWMS version must link to a task	Required for swms_versions.task_id
Activity steps must link to SWMS version	Required for swms_activity_steps.swms_version_id
Pre-start questions must link to task	Required for prestart_questions.task_id
Industry and occupation mapping must be valid	Required for access control
Runtime tables must not be imported from Excel	They should be generated by app use
Import batches must be recorded	Needed for traceability
Failed validation must stop import	Prevents bad data entering PostgreSQL

The main rule is: AI generates content, Python controls the Excel structure, and the Laravel importer protects the database.

This section defines the BA/DA-level validation requirements. After Kevin provides sample workbooks and the first importer scope is confirmed, these requirements can be converted into detailed JSON Schema rules, Python validation checks, and Laravel importer logic.

Overall Conclusion

The v0.3 database migration is a strong foundation for the OpsFortress / WHS App workflow. The local database has been created successfully, the Laravel migrations ran successfully, and the main P0 areas are covered. The core master-data and content relationships are working, and the database now shows a clear boundary between imported content, app-generated runtime data and importer-generated tracking records.

The main limitation is runtime validation. The runtime and evidence tables exist, but they currently contain no seed data. These will need to be tested later using demo runtime data or real worker app activity.

Key Findings:

Area	Result	Note
Local migration	Pass	Laravel migrations successfully ran
Table coverage	Pass	48 tables created
P0 coverage	Pass	Core P0 tables are present
P1 coverage	Present	Post-task and training tables already exist
Core relationships	Pass	Master/content relationships are connected
Runtime data	Not yet validated	Runtime/evidence tables currently have no rows
Importer tracking	Ready structurally	Tracking tables exist; importer has not processed a workbook yet
Python / JSON workflow	Recommended	Best path for stable workbook generation and validation

Open Point	Reason
Confirm audit_events design	Need to know whether audit_events uses a generic reference model or needs direct worker_task_session linkage
Add demo runtime data	Needed to test the full worker journey
Confirm first importer scope	Need to know whether first demo is P0 only or includes post-task/training

Review Kevin sample workbooks	Needed to compare sheet names, column order, IDs, required fields and relationship structure
Define first locked Excel framework	Needed for Python generation and importer validation
Convert Part 6 requirements into implementation rules	Needed for JSON Schema, Python checks and Laravel importer logic
Recommended Next Step	Owner / Purpose
Confirm open database design points with Yiming	Database and audit traceability alignment
Use Kevin sample workbooks for comparison	Validate sheet names, columns, IDs and relationships
Define the first importer-ready Excel framework	Create the Python output target
Build Python validation rules	Enforce framework and database requirements
Build or confirm Laravel importer command	Load validated Excel into PostgreSQL
Test one full workbook import	Confirm end-to-end import flow
Add demo runtime data	Validate worker action -> event record -> evidence/audit flow